

lecture_10.py

Explain what makes LLM inference hard/expensive (especially generation), and survey the main systems + model techniques that improve latency (user speed) and throughput (service speed).



```

1 from sympy import symbols, oo
2 from execute_util import text, link, image
3 from lecture_util import article_link
4 from references import Reference, llama3, gqa, mla, longformer, sparse_transformer, mistral_7b
5
6 # Define symbols corresponding to the shape of the Transformer model
7 B, S, T, D, F, N, K, H, L, V = symbols("B S T D F N K H L V", positive=True)
8 c = symbols("c", positive=True) # Just a constant that helps with taking limits
9 memory_bandwidth = symbols("memory_bandwidth", positive=True)
10
11 scaling_book_transformers = Reference(title="[Scaling book chapter on Transformers]", url="https://jax-
ml.github.io/scaling-book/transformers/")
12 scaling_book_inference = Reference(title="[Scaling book chapter on Transformers]", url="https://jax-ml.github.io/scaling-
book/inference/")

```

13

14 def main():

Inference: given a **fixed model**, generate responses given prompts

推理inference: 模型训练完了, 你给它一个提示词 prompt, 它开始一个 token 一个 token 地生成回复。

16

17

Understanding the inference workload

```

18 landscape()
19 review_transformer()
20 review_of_arithmetic_intensity()
21 arithmetic_intensity_of_inference()
22 throughput_and_latency()

```

23

24

Taking shortcuts (lossy)

```

25 reduce_kv_cache_size()
26 alternatives_to_the_transformer()
27 quantization()
28 model_pruning()

```

29

Summary: reduce inference complexity without hurting accuracy

31

From scratch recipe:

1. Define faster model architecture
2. Train faster model

35

Distillation recipe:

1. Define faster model architecture
2. Initialize weights using original model (which has a different architecture)
3. Repair faster model (distillation)

40

41

Use shortcuts but double check (lossless)

```

42 speculative_sampling()

```

43

44

Handling dynamic workloads

Batching over sequences in live traffic is tricky because:

1. Requests arrive at different times (waiting for batch is bad for early requests)
2. Sequences have shared prefixes (e.g., system prompts, generating multiple samples)
3. Sequences have different lengths (padding is inefficient)

`continuous_batching()`

`paged_attention()`

Summary

- Inference is important (actual use, evaluation, reinforcement learning)
- Different characteristics compared to training (memory-limited, dynamic)
- Techniques: new architectures, quantization, pruning/distillation, speculative decoding
- Ideas from systems (speculative execution, paging)
- New architectures have huge potential for improvement

`def landscape():`

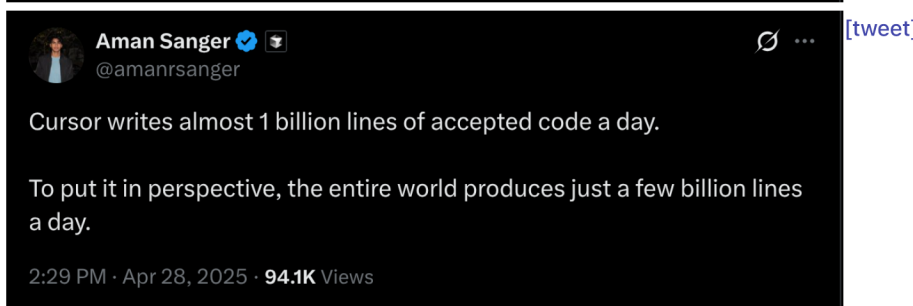
Inference shows up in many places:

- Actual use (chatbots, code completion, batch data processing)
- Model evaluation (e.g., on instruction following)
- Test-time compute (thinking requires more inference)
- Training via reinforcement learning (sample generation, then score)

推理不仅是聊天：产品线上服务、模型评测、test-time “思考”、RL

训练采样都要大量推理。训练只花一次钱，但推理要反复花钱，所以必须优

Why **efficiency** matters: **training is one-time cost, inference is repeated many times**



Metrics:

- Time-to-first-token (TTFT): how long user waits before any generation happens (matters for interactive applications)
- Latency (**seconds/token**): **how fast tokens appear for a user** (matters for interactive applications)
- Throughput (**tokens/second**): useful for batch processing applications

TTFT 首 token 时间：用户多久看到第一个字。
延迟（每 token 秒数）：字“蹦出来”的速度。
吞吐（每秒 token 数）：服务器总体产出能力。

Key considerations in efficiency:

- Training (supervised): you see all tokens, can **parallelize** over sequence (matmul in Transformer)
- Inference: you have to generate **sequentially**, can't parallelize, so harder to fully utilize compute

Companies doing inference (a big deal for anyone who has a product or platform):

- Providers serving closed models (OpenAI, Anthropic, Google, etc.)
- Providers serving open-weight models (Together, Fireworks, DeepInfra, etc.)

训练时你“同时看到整段文本”，可以并行算；推理生成必须一步一步来（自回归），并行性差，更难把 GPU 吃满。

Open-source packages:

B: batch size (一次并行处理多少条序列) L: 层数 (多少个 Transformer blocks) T: query 的序列长度 (本次要算的 token 数) S: key/value 的序列长度 (可被注意力看的历史长度) V: 词表大小 vocab D: 模型隐藏维度 d_model (embedding/hidden size) F: MLP 隐藏层维度 (通常比 D 大, 比如 4D) H: 每个 head 的维度 (head_dim) N: query heads 数 (Q 有多少个头) K: key/value heads 数 (K/V 有多少个头, GQA 会让它小于 N) G: 每个 kv head 对应多少个 q head, $G = N / K$

- vLLM (Berkeley) [talk]
- Tensor-RT (NVIDIA) [article]
- TGI (Hugging Face) [article]

```
def review_transformer():  
    [Scaling book chapter on Transformers]
```

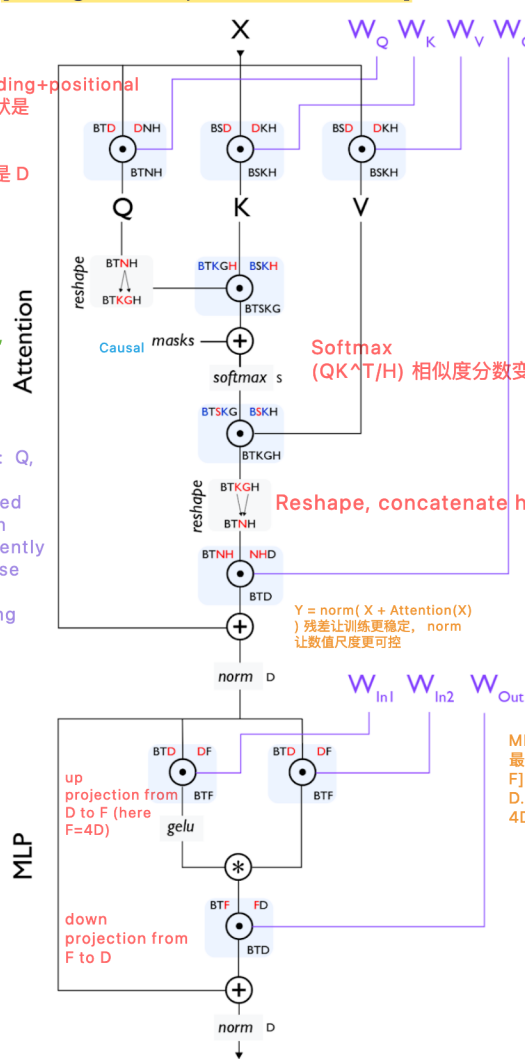
Input matrix X 是一批序列的 embedding + positional encoding 的表示, 形状是 [B, T, D]: -B 条序列, -每条取 T 个 token, -每个 token 是 D 维向量。

Input X is multiplied /linearly projected by three weight matrices (W_q, W_k, W_v): $Q = X W_q$ -> Query $K = X W_k$ -> Key $V = X W_v$ -> Value

Reshaping for Cache: Q, K, and V need to be reshaped and managed (e.g., paged attention uses blocks) to efficiently store and access these cached K/V pairs, especially with varying sequence lengths.

Softmax: 对每个 query, 沿着 key 维度 S 做 softmax 得到注意力权重 (概率分布)。

$M = \text{GeLU}(A) \odot B$ 是逐元素乘法 $\text{GeLU}(A)$ 像“开关强度”, 乘以 B 像“把信息通过开关过滤”。 gated MLP (门控前馈网络)



这里只显示了一层 transformer block. 每一层 (Transformer block) 都有自己的一套参数: $W_Q(\ell), W_K(\ell), W_V(\ell), W_{up}(\ell), W_{down}(\ell)$. training 的时候参数会 update, inference 的时候只读参数, 不 update.

symbol	dimension
B	batch
L	number of layers
T	sequence length (query)
S	sequence length (key value)
V	vocab
D	d_model, embedding dimension
F	MLP hidden dimension
H	d_k
N	number of query heads
K	number of key/value heads
G	q heads per kv head = $N // K$

Example: 1. Input (Batch, SeqLen, EmbedDim) 2. Project to Q, K, V (e.g., (Batch, Seq

MLP 两个线性层投影 (W_{in1}, W_{in2}), 中间有 gelu 和一个乘号 // 门控, 最后再用 W_{out} 从 F 投回 D. Two up projections: $A = X @ W_{in1} \rightarrow [B, T, F]$ $B = X @ W_{in2} \rightarrow [B, T, F]$ Then down projections from F back to D. 这里的 F 通常比 D 大 (常见是 4D), 目的: 给模型更强的非线性表达能力。

一个 Transformer block: 输入 X(B, T, D) -> 先做多头注意力 (Q/K/V, mask, softmax, 加权 V、拼头、 W_o) + 残差相加归一化 -> 再开门控 MLP (D -> F -> D) + 残差相加归一化。This diagram shows one layer of a standard Transformer and flows from top-to-bottom. We use a single-letter convention to describe the shapes and layouts of arrays in a Transformer, again showing contracting dimensions in red, and batched dimensions in blue. In a given operation, the input shape is given on top-left and the parameter shape is given on the top-right, with the resulting shape below, e.g. BTD is the input shape for the gating einsum and DF is the weight shape.

$F = 4 * D$ MLP (FFN) 中间层维度 通常是 D 的 4 倍 这是 GPT / BERT 的默认设定。 - Attention 是“信息混合” - MLP 是“非线性表达能力的助力” - 扩大维度 = 提升表达能力
 $D = N * H$ (多头拆分) 模型维度
 $D = \text{头数} \times \text{每个头的维度}$ Example: $D = 4096$ $N = 32$ heads $H = 128$

Simplifications (following conventions): $F = 4 * D, D = N * H, N = K * G, S = T$
FLOPs for a feedforward pass: $6 * (B * T) * (\text{num_params} + O(T))$

$N = K * G$ (GQA 关系) Each KV head is shared by G query heads. 在 GQA 中: Query 头很多 (N) Key/Value 头更少 (K) 每个 KV 头服务 G 个 Q 头 这是为了减少 KV cache / FLOPs / memory, 对 inference 很关键。 $Y = \text{concat}(\text{heads}) * W_o$.

Self-attention 里: $S = T$ Keys and queries come from the same sequence. Query 长度 = Key, Value 长度 如果是 cross-attention 才会 $S \neq T$

One Feedforward Pass: from input to output No backprop, no gradient/derivatives. 1. Attention - compute Q/K/V linear projections - compute attention (QK^T, softmax, weighted sum V) - output projection W_o (D by D matrix): $Y = \text{concat}(\text{heads}) * W_o$. 2. MLP - MLP/FFN (up-projection, activation/gating, down-projection)

```
def review_of_arithmetic_intensity():  
    Setup: multiply X (B x D) and W (D x F) matrix  
    Intuition: B is batch size, D is hidden dimension, F is up-projection dimension in MLP  
  
    Let's do FLOPs and memory read/write accounting for the matrix multiplication (X * W).  
    flops = 0  
    bytes_transferred = 0  
  
    Steps:  
    1. Read X (B x D) from HBM  
    bytes_transferred += 2*B*D  
    2. Read W (D x F) from HBM  
    bytes_transferred += 2*D*F  
    3. Compute Y = X (B x D) @ W (D x F)  
    flops += 2*B*D*F
```

Generation阶段 很多场景B很小例如只有一个到几十个用户
一次生成1个 token, 下一个 token
必须等上一个算完(Autoregression) GPU
算得并不多, 但要处理很多数据, 所以 GPU
大部分时间在“等内存”, 而不是在算。比如B=1,
H100需要295才行, intensity相差了快300倍 Tensor
core:等数据 HBM带宽: 满负荷 This is memory limit

```

113 4. Write Y (B x F) to HBM
114 bytes_transferred += 2*B*F
115
116 Let's take stock of the accounting results.
117 assert flops == 2*B*D*F
118 assert bytes_transferred == 2*B*D + 2*D*F + 2*B*F
119 Recall that arithmetic intensity is how much compute we do per byte transferred (want to be high).
120 intensity = (flops / bytes_transferred).simplify() # @inspect intensity
121
122 Assuming B is much less than D and F, then we can simplify:
123 intensity = intensity.subs(D, c*B).subs(F, c*B).limit(c, oo).simplify() # @inspect intensity
124 assert intensity == B
125
126 Accelerator intensity of H100:
127 flops_per_second = 989e12 峰值算力 flops/s
128 memory_bandwidth = 3.35e12 内存带宽 bytes/s
129 accelerator_intensity = flops_per_second / memory_bandwidth # @inspect accelerator_intensity
130 assert round(accelerator_intensity) == 295
131
132 If computation intensity > accelerator intensity, compute-limited (good)
133 If computation intensity < accelerator intensity, memory-limited (bad)
134 Conclusion: compute-limited iff B > 295
135
136 Extreme case (B = 1, corresponding to matrix-vector product):
137 • Arithmetic intensity: 1
138 • Memory-limited (read D x F matrix, perform only 2DF FLOPs)
139 • This is basically what happens with generation...

```

H100 希望你每读 1 byte, 算 ~295 次 MLP
的算术强度 = batch size B generation 时
B 很小 → 内存受限 所有 inference
优化, 本质都在: 减少内存访问 / KV cache
/ 权重读取

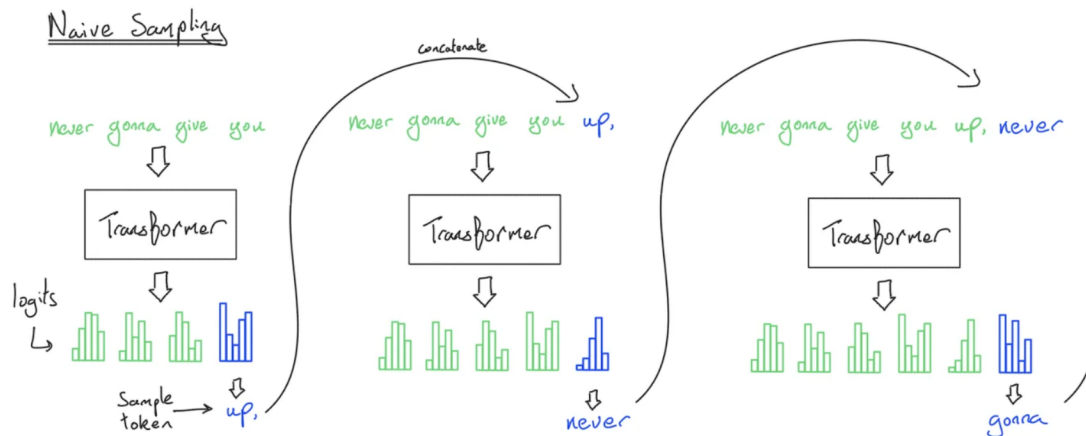
Proof: when $c \rightarrow +\infty$, the limit of intensity is Batch size B. $\text{intensity} = \text{flops}/\text{bytes_transferred} = 2BDF/(2BD+2DF+2BF) = 2BcBcB/(2BcB+2cBcB+2BcB) = Bc/(c+2) \rightarrow B$ when $c \rightarrow +\infty$

GPU做任何任务都分两步: 1. 从内存里把数据拿出来 2. 对数据做数学运算 所以速度慢只可能有两个原因: 1. 算得不够快 → 算力受限compute-limited。算力是瓶颈。ALU / Tensor Core 一直在忙。内存带宽还没用满。多给一点计算单元, 程序就会更快。 2. 数据来得太慢 → 内存受限memory-limited。GPU 在等数据, 内存是瓶颈。计算单元大量空闲。GPU 一直在“等数据从 HBM 送过来”。再强的算力也没用。 H100 想利用好compute而不是空等, 每从内存读 1 个 byte, 最好能做 ≈ 295 次浮点运算

```

142 def arithmetic_intensity_of_inference():
143     [Scaling book chapter on Transformers]
144
145

```



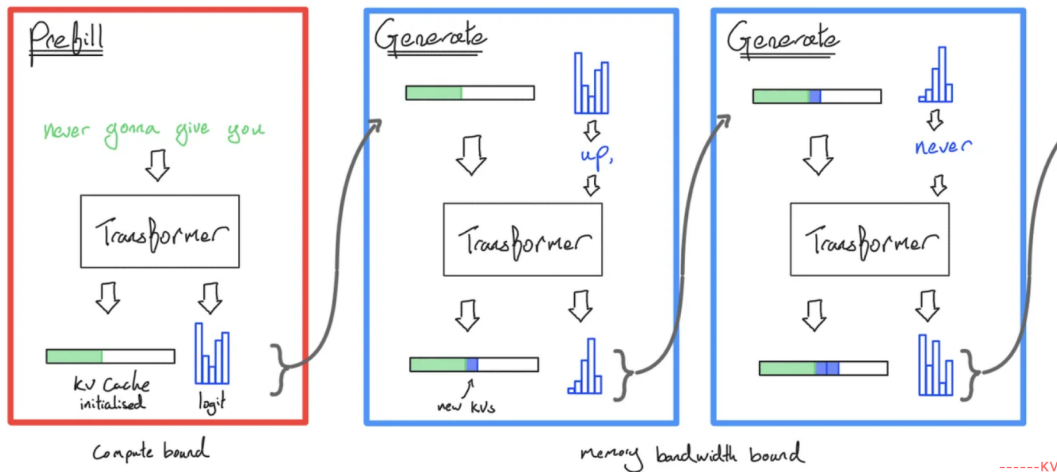
Naive inference: to generate each token, feed history into Transformer

Complexity: generating T tokens requires $O(T^3)$ FLOPs (one feedforward pass is $O(T^2)$) T: query sequence length

Observation: a lot of the work can be shared across prefixes

Solution: store **KV cache** in HBM

151

Sampling with KV cache

KV cache: for every sequence (B), token (S), layer (L), head (K), store an **H-dimensional vector**

Prefill: 因为 prompt 的 token

都已知, 所以可以像训练一样并行处理 (矩阵乘法很大, 很容易跑满 GPU).

similar to training, use teacher forcing + causal mask.

Two stages of inference:

1. **Prefill**: given a prompt, encode into vectors (**parallelizable** like in training)
2. **Generation**: generate new response tokens (**sequential**)

157 Generation: 生成回答时, 新 token 依赖上一个 token, 所以必须一步一步生成。每一步只生成很少 token (常常 T=1)。串行。

Let's compute the FLOPs and memory IO for both the MLP and attention layers.

S is the number of tokens we're conditioning on, **T** is the number of tokens we're generating.

Later, we'll specialize to **prefill (T = S)** and **generation (T = 1)**.

MLP layers (only looking at the matrix multiplications)

flops = 0

bytes_transferred = 0

Steps:

1. **Read** X (B x T x D) from HBM

bytes_transferred += 2*B*T*D

这里假设张量用的是 bf16 / fp16 (主流), 每个数占 2 个 byte. fp32 (4 bytes) 太大、太慢。

2. **Read** Wup (D x F), Wgate (D x F), Wdown (F x D) from HBM

bytes_transferred += 3 * 2*D*F

注意: Wup, Wgate, Wdown don't depend on B.

3. **Compute** U = X (B x T x D) @ Wup (D x F)

flops += 2*B*T*D*F

Gated MLP: gelu, SwiGLU

4. **Write** U (B x T x F) to HBM

bytes_transferred += 2*B*T*F

5. **Compute** G = X (B x T x F) @ Wgate (D x F)

flops += 2*B*T*D*F

D

6. **Write** G (B x T x F) to HBM

bytes_transferred += 2*B*T*F

7. **Compute** Y = GeLU(G)*U (B x T x F) @ Wdown (F x D)

flops += 2*B*T*D*F

8. **Write** Y (B x T x D) to HBM

bytes_transferred += 2*B*T*D

Let's take stock of the accounting results.

assert flops == 6*B*T*D*F

assert bytes_transferred == 4*B*T*D + 4*B*T*F + 6*D*F

intensity = (flops / bytes_transferred).simplify() # @inspect intensity

Assume that B*T is much smaller than D and F.

intensity = intensity.subs(D, c*B*T).subs(F, c*B*T).limit(c, oo).simplify() # @inspect intensity

assert intensity == B*T

Proof: intensity = 6BTDF/(4BTD+4BTF+6DF)
= 6BTcBTcBT/(4BTcBT+4BTcBT+6cBTcBT)
6BTc/(4+4+6c) limit -> BT when c-> +∞

asymptotic analysis when c -> +∞ 不是在说现实里 c

= 真的无限大 而是在问: 当模型足够大时, 哪些项决定性能、哪些项可以忽略。注: 某些

-----KV cache

的形状定义----- B: 有多少条并发序列 (batch) S: 每条序列 token (conditioning context) L: 层数 K: KV heads 数 (注意这里用 K, 不一定等于 query heads N, 这是 GQA/MQA 的关键) H: 每个 head 的维度 -----KV cache 存的是----- 对每一层、每个 KV head、每个历史 token, 都存一个 H 维的 K 向量 + H 维的 V 向量。 (Paged Attention 存的不是固定的 H 维向量)

=====Inference 本质问题===== “生成慢, 不是因为算得慢, 而是因为每生成一个字, 都要从显存里翻一整本书。”
=====优化的方向===== 方向1. 让同一份数据多用几次 batching, 增大 B (并发序列数) 把多个请求拼在一起, 一次读 W, 给 B 个 token 很难做到几百。
方向2. 让要读的数据变少 #1 减少权重读取 Quantization (量化) fp16 -> int8 / int4 权重体积 ↓ 4-8 倍 同样算 2DF FLOPs 但 bytes ↓ 算术强度 ↑
----- #2 减少 KV cache 访问 (更关键) generation 时 attention 要: 读所有历史 token 的 K/V KV cache 随 T 线性增长 ▶ GQA / MQA 减少 KV head 数 KV cache ↓ N/K 倍 内存访问 ↓ ▶ MLA / 压缩 KV K/V 投影到低维 latent cache 更小 ----- #3 减少 “无用 attention” Sliding window / local attention 不看全部历史 只看最近一段 -----
/ fused kernels 把: 读 Q/K/V softmax 加权 融合成一个 kernel 减少中间结果反复进出 HBM ▶ PagedAttention (vLLM) KV cache 分块 减少碎片 提高 cache 命中率

For the two stages:

1. Prefill: easy to make compute-limited (good) by making B T large enough

2. Generation:

Prefill: $T=S$. 通常 S 很大 (比如 1k token), 所以 BT 可以很大 $\rightarrow$ 容易 compute-limited。Generation: $T=1$, 这时 $BT=B$ 。B 是并发请求数, 在真实在线系统里很难做到几百 (否则延迟大) $\rightarrow$ 难以 compute-limited!

• Generating one token at a time ($T=1$)

• B is number of concurrent requests, hard to make large enough!

Attention layers (focusing on the matrix multiplications with FlashAttention)

flops = 0

$F = 4 \times D$, $D = N \times H$, $N = K \times G$, $S = T$

bytes_transferred = 0

T: sequence length-Query S:
sequence length-Key, Value

Steps:

1. Read Q ($B \times T \times D$), K ($B \times S \times D$), V ($B \times S \times D$) from HBM 注意: Q, K, V depends on B.

bytes_transferred += $2 \times B \times T \times D + 2 \times B \times S \times D + 2 \times B \times S \times D$

2. Compute $A = Q (B \times T \times D) @ K (B \times S \times D)$

flops += $2 \times B \times S \times T \times D$

3. Compute $Y = \text{softmax}(A) (B \times S \times T \times K \times G) @ V (B \times S \times K \times H)$

flops += $2 \times B \times S \times T \times D$

4. Write Y ($B \times T \times D$) to HBM

bytes_transferred += $2 \times B \times T \times D$

B: batch

size (一次并行处理多少条序列) L: 层数 (多少个

Transformer blocks) T: query

的序列长度 (本次要算的 token 数) S: key/value

的序列长度 (可被注意力看的历史长度) V: 词表大小

vocab D: 模型隐藏维度

d_model (embedding/hidden size) F: MLP

隐藏层维度 (通常比 D 大, 比如 4D) H: 每个 head

的维度 (head_dim) N: query heads 数 (Q

有多少个头) K: key/value heads 数 (K/V

有多少个头, GQA 会让它小于 N) G: 每个 kv head

对应多少个 q head, $G = N / K$

assert flops == $4 \times B \times S \times T \times D$

assert bytes_transferred == $4 \times B \times S \times D + 4 \times B \times T \times D$

intensity = (flops / bytes_transferred).simplify() # @inspect intensity

assert intensity == $S \times T / (S + T)$

Intensity = $4 \times B \times S \times T \times D / (4 \times B \times S \times D + 4 \times B \times T \times D)$
= $ST / (S + T)$

For the two stages:

1. Prefill: $T = S$

prefill_intensity = intensity.subs(T, S).simplify() # @inspect prefill_intensity

assert prefill_intensity == $S/2$ # Good!

2. Generation: $T = 1$

generate_intensity = intensity.subs(T, 1).simplify() # @inspect generate_intensity

assert generate_intensity < 1 # Bad!

1. $2 \times V \times D$: Embedding 相关参数 V:

词表大小 (vocab) D:

模型维度 (d_model) 输入

embedding ($V \rightarrow D$): $V \times D$ 输出 logits

的权重 (unembedding/lm head,

$D \rightarrow V$, outside a transformer block right

before output): 也近似

$V \times D$ 所以合起来近似: $2VD$

Unlike MLPs, no dependence on B, so batching doesn't help!

Why?

• In MLP layers, every sequence hits the same MLP weights (Wup, Wgate, Wdown don't depend on B)

• In attention layers, every sequence has its own vectors KV cache (Q, K, V all depend on B)

2. $D \times F \times 3 \times L$: MLP/FFN 参数 每一层

Transformer 的 gated MLP 有 3

个大矩阵: Wup: $D \rightarrow F$: 参数

$D \times F$ Wgate: $D \rightarrow F$: 参数 $D \times F$ Wdown:

$F \rightarrow D$: 参数 $F \times D$ 共有 L 层: $D \times F \times 3 \times L$

Summary prefill 往往算力受限; 生成往往内存受限

• Prefill is compute-limited, generation is memory-limited

• MLP intensity is B (requires concurrent requests), attention intensity is 1 (impossible to improve)

```
def compute_transformer_stats(config): # @inspect config
```

"""Return symbols corresponding to various statistics of a Transformer."""

The memory, throughput, and latency depends on the shape of the Transformer.

Compute the number of parameters in the Transformer:

num_params = $2 \times V \times D + D \times F \times 3 \times L + (2 \times D \times N \times H + 2 \times D \times K \times H) \times L$

To store parameters, just use bf16 (training requires fp32)

parameter_size = num_params * 2 # 2 for bf16

3. Attention 的投影矩阵参数 在 attention

里常见四个投影矩阵: W_Q, W_K, W_V, W_O 1).

$2 \times D \times N \times H$: Q 和 O (输出) 投影: W_Q: 把 D

投影到 $N \times H = D$, 参数量 $D \times (NH)$ W_O: 把

$NH = D$ 投影到 D, 参数量

$(NH) \times D$ 相加 = $2DNH$ 2). $2 \times D \times K \times H$: K 和

V 投影 (依赖 K) W_K: 参数

$D \times (KH)$ W_V: 参数

$D \times (KH)$ 相加 = $2DKH$ 最后乘以 L

层: $(2DNH + 2DKH) \times L$

We also don't need gradients and optimizer states since we're not training.

But we do have to store the KV cache (which are some of the activations) for each sequence (of length S):

How much we have to store per sequence:

kv_cache_size = $S \times (K \times H) \times L \times 2 \times 2$ # 2 for key + value, 2 for bf16

KV cache 存的是, 每一层、每一个 KV head、每一个历史 token 的 Key 和 Value 向量

```

247 Total memory usage:
248 memory = B * kv_cache_size + parameter_size
249 Latency is determined by memory IO (read all parameters and KV cache for each step)
250 latency = memory / memory_bandwidth
251 Throughput is the inverse of latency, but we're generating B tokens in parallel
252 throughput = B / latency
253
254 # Substitute
255 num_params = num_params.subs(config).simplify() # @inspect num_params
256 memory = memory.subs(config).simplify() # @inspect memory
257 latency = latency.subs(config).simplify() # @inspect latency
258 throughput = throughput.subs(config).simplify() # @inspect throughput
259
260 return num_params, memory, latency, throughput
261
262 def llama2_13b_config(args={}):
263     return {S: 1024, D: 5120, F: 13824, N: 40, K: 40, H: 128, L: 40, V: 32000, memory_bandwidth: 3.35e12, **args}
264
265 def throughput_and_latency():
266     So we have shown that inference is memory-limited.
267     Let us now compute the theoretical maximum latency and throughput of a single request.
268     Assumption: can overlap compute and communication perfectly and ignore various types of overhead.
269
270     Instantiate latency and throughput for Llama 2 13B on an H100:
271     config = llama2_13b_config()
272     num_params, memory, latency, throughput = compute_transformer_stats(config)
273
274     If we use a batch size of 1:
275     bs1_memory = memory.subs(B, 1).simplify() # @inspect bs1_memory
276     bs1_latency = latency.subs(B, 1).simplify() # @inspect bs1_latency
277     bs1_throughput = throughput.subs(B, 1).simplify() # @inspect bs1_throughput
278
279     If we use a batch size of 64 (worse latency, better throughput):
280     bs64_memory = memory.subs(B, 64).simplify() # @inspect bs64_memory
281     bs64_latency = latency.subs(B, 64).simplify() # @inspect bs64_latency
282     bs64_throughput = throughput.subs(B, 64).simplify() # @inspect bs64_throughput
283
284     If we use a batch size of 256:
285     bs256_memory = memory.subs(B, 256).simplify() # @inspect bs256_memory
286     bs256_latency = latency.subs(B, 256).simplify() # @inspect bs256_latency
287     bs256_throughput = throughput.subs(B, 256).simplify() # @inspect bs256_throughput
288     Doesn't fit into memory, but throughput gains are diminishing too...
289
290     Tradeoff between latency and throughput:
291     1. Smaller batch sizes yields better latency but worse throughput
292     2. Larger batch sizes yields better throughput but worse latency
293
294     Easy parallelism: if you launch M copies of the model, latency is the same, throughput increases by M!
295     Harder parallelism: shard the model and the KV cache [Scaling book chapter on Transformers]
296
297     Note: time-to-first-token (TTFT) is essentially a function of prefill
298     Use smaller batch sizes during prefill for faster TTFT
299     Use larger batch sizes during generation to improve throughput
300
301     KV cache 会不会把 W_K / W_V 改掉, KV cache
302     存的是: K = X · W_K, V = X · W_V, X
303     变, W_K和W_V不变
304     Recall that memory is the bottleneck for inference.
305     So let's try to reduce the size of the KV cache

```

假设 inference 是 memory-limited, 所以每步时间="要从显存(HBM)搬多少数据 / 显存带宽"。然后由此推出 latency/throughput 的 batch tradeoff。 - parameter_size: 模型权重总大小 (全局共享, 一份就够) - kv_cache_size: 每条序列的 KV cache 大小 (随序列长度 S、层数 L、KV heads K、head_dim H 增长) - B: 并发序列数, 所以 KV cache 需要 B 份

这是上限估算的保守/粗化假设。权重 (parameters): 每步确实要用到很多层的权重做 matmul, 但部分可能被 cache 命中、也可能重复利用; 并且不同 kernel 的访存模式不同。KV cache: 每步 attention 必须读取"当前需要注意的那些历史 token 的 K/V", 所以 KV 的读取是硬需求, 尤其 S 很大时非常显著。每步生成都要消耗大量 HBM 带宽, KV cache 是不可避免的带宽开销

用 Llama2-13B 的典型结构参数 (S=1024 上下文) H100 的 HBM 带宽约 3.35 TB/s (用字节/秒表示) 算出理论上限的 memory/latency/throughput

latency ≈ memory / memory_bandwidth 背后的核心假设: (忽略 overhead) - 计算可以和数据搬运完美重叠 - 主 + 读全部 KV", 有缓存/L2/量子融合/flash decoding 等优化。但用这个近似能抓住趋势: 带宽越大越快, 越大越慢。

B=1: 最快响应 (最好 latency), 但吞吐低 B=64: 响应慢一些, 但每秒生成 token 总量更高 B=256: 可能显存放不下 (KV 太大), 而且吞吐提升也开始变小 B 越大, memory 越大 → latency ≈ memory/bw 越大 (更慢) 虽然 latency 变大, 但分子也变大 → 吞吐通常会上升, 但会出现递减收益 (diminishing returns)

Easy parallelism: 复制 M 份模型 (data parallel serving) - 每个 GPU 各自服务一部分请求 - 单请求 latency 不变 - 总 throughput 近似 ×M Harder parallelism: 把一个模型切开 (shard) - 把权重/层/张量切到多卡 - KV cache 也要分布式管理 - 通信复杂、调度复杂, 但能跑更大模型或更长上下文

B=256放不下

Reduce KV cache: GQA / MQA, MLA, cross-layer attention, local/hybrid attention.

...but make sure we don't lose too much accuracy.

GQA Intuition: Keys/Values define what information exists. Queries define how you look at it. Multiple people (queries) can ask different questions about the same database (keys/values). You don't need to duplicate the database for every person.

Grouped-query attention (GQA)

Query heads = Key heads
[Ainslie+ 2023] **= Value heads = N**

$K \ll N$

Extreme case: $K=1$

推理 (尤其 generation) 被 KV cache 的“显存占用 + 显存带宽读写”卡住了。GQA 的核心就是：减少 KV head 数 K ，从而把 KV cache 变小 (按 N/K 成比例缩小)，在基本不掉精度的

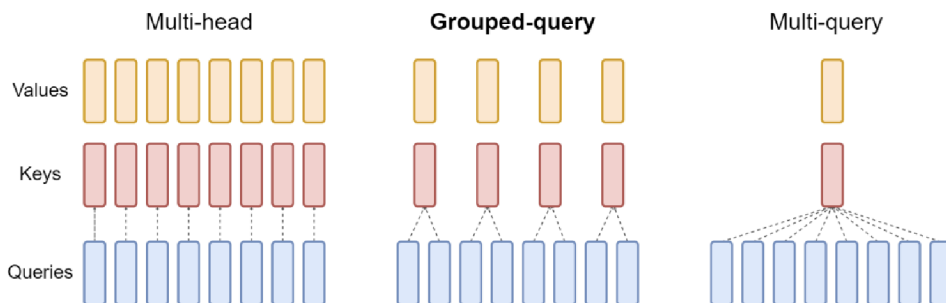


Figure 2: Overview of grouped-query method. Multi-head attention has H query, key, and value heads. Multi-query attention shares single key and value heads across all query heads. Grouped-query attention instead shares single key and value heads for each *group* of query heads, interpolating between multi-head and multi-query attention.

Idea: N query heads, but only K key and value heads, each interacting with N/K query heads

Multi-headed attention (MHA): $K=N$

Multi-query attention (MQA): $K=1$

Group-query attention (GQA): K is somewhere in between

Latency/throughput improvements:

N = number of query heads K = number of key/value heads $G = N / K$ = queries per KV head Each KV head is shared by G query heads. For each query head i , attention _{i} ($Q_i, K_{g(i)}, V_{g(i)}$) where $g(i)$ maps query heads to a KV head. Queries stay independent. Keys/values are reused. Without GQA: KV cache $\sim B \cdot T \cdot N \cdot H$ With GQA: KV cache $\sim B \cdot T \cdot K \cdot H$ Since $N \ll K$: Memory $\downarrow$ Bandwidth $\downarrow$ Throughput $\uparrow$ Longer context possible

[Ainslie+ 2023]

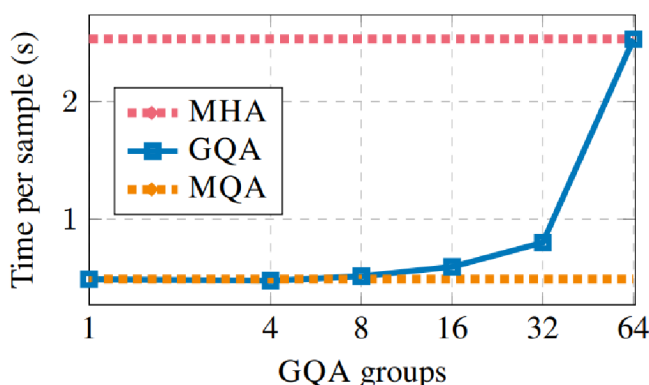


Figure: Time per sample for GQA-XXL as a function of the number of GQA groups with input length 2048 and output length 512. Going from 1 (MQA) to 8 groups adds modest inference overhead, with increasing cost to adding more groups.

Reduce the KV cache by a factor of N/K **$kv_cache_size = S \cdot (K \cdot H) \cdot L \cdot 2 \cdot 2$**

```
config = llama2_13b_config({K: 40, B: 64}) # Original Llama 2 13B
```

```
k40_num_params, k40_memory, k40_latency, k40_throughput = compute_transformer_stats(config) # @inspect k40_memory,
```

```
@inspect k40_latency, @inspect k40_throughput
```

```
config = llama2_13b_config({K: 8, B: 64}) # Use GQA with 1:5 ratio
```

```
k8_num_params, k8_memory, k8_latency, k8_throughput = compute_transformer_stats(config) # @inspect k8_memory,
```

```
@inspect k8_latency, @inspect k8_throughput
```

This also means we can use a larger batch size:

```
config = llama2_13b_config({K: 8, B: 256}) # Increase batch size
```

```
k8_bs_num_params, k8_bs_memory, k8_bs_latency, k8_bs_throughput = compute_transformer_stats(config) # @inspect
```

```
k8_bs_memory, @inspect k8_bs_latency, @inspect k8_bs_throughput
```

Worse latency, but better throughput (and it fits in memory now!). 现在 $B=256$ 可以放下了，之前不可以

Check that accuracy doesn't drop: [Ainslie+ 2023]

Takeaway: 1. GQA: N 个 Q 头不变, K 个 KV 头减少 ($K < N$) 2. KV cache 大小 $\propto K$, 所以节省因子 = N/K 3. KV cache 变小 $\rightarrow$ 显存占用更小 + 每步带宽更小 $\rightarrow$ generation 更快, 还能支持更大的 batch

K 太小 (接近 MQA) 共享太强, 可能损失表达力 适中的 K (GQA) 通常能保持精度, 同时显著省 KV

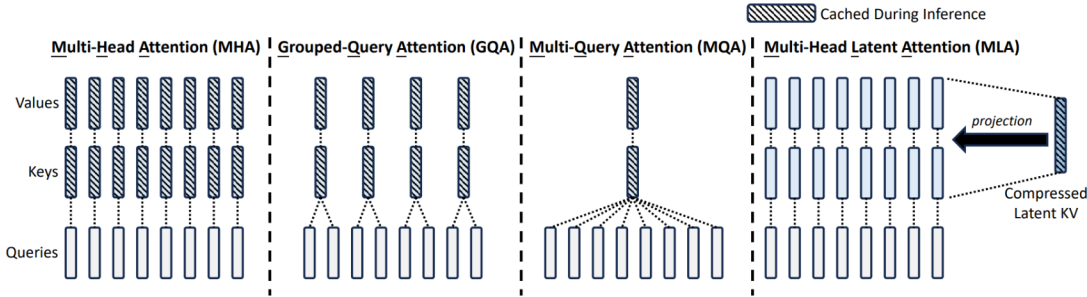
Model	$F1$	T_{infer}	Average	CNN	arXiv	PubMed	MediaSum	MultiNews	WMT	TriviaQA
		s		R_1	R_1	R_1	R_1	R_1	BLEU	F1
MHA-Large	0.37	46.0	42.9	44.6	46.2	35.5	46.6	27.7	78.2	
MHA-XXL	1.51	47.2	43.8	45.6	47.5	36.4	46.9	28.4	81.9	
MQA-XXL	0.24	46.6	43.0	45.0	46.9	36.1	46.5	28.5	81.3	
GQA-8-XXL	0.28	47.1	43.5	45.4	47.7	36.3	47.2	28.4	81.6	

MHA (多头 attention) : 模型更大 → 精度更高 47.2. 但 KV cache 太大, 推理速度爆炸式变慢, 1.51s。MQA-XXL (极端压缩 KV) : KV cache 最小 → 推理最快, 但 共享过度 → 精度有可见下降 GQA-8-XXL: 速度: 接近 MQA, 精度: 几乎等于 MHA-XXL

MHA: 每个 Q head 都有自己独立的 K/V 能学到非常细粒度、差异化的注意力模式 MQA: 所有 Q head 共享同一套 K/V 表达能力被严重压缩 GQA: Q head 分组共享 K/V 表达力介于两者之间 GQA 是“工程最优解”, 但不是“表达最优解”

Multi-head latent attention (MLA)

保留 MHA 的表达力, 又拥有比 GQA / MQA 更低的 KV cache 成本? Multi-Head Latent Attention (MLA): 不减少 head 数, 而是把“完整的 KV 信息”压缩到一个低维 latent 空间里。



缓存的是: Compressed Latent KV 真正做 attention 前: 再从 latent KV 投影回每个 head 需要的空间 存的是低维 用的时候再展开 这 MLA 比 GQA 更“聪明”的地方。

Key idea: project down each key and value vector from $N \times H$ dimensions to C dimensions

DeepSeek v2: reduce $N \times H = 16384$ to $C = 512$ 压缩32倍

Wrinkle: MLA is not compatible with RoPE, so need to add additional 64 dimensions for RoPE, so $512 + 64 = 576$ total dimensions

Latency/throughput improvements follow similarly from the KV cache reduction as argued earlier

Let's now check the accuracy.

First, MHA is better than GQA (though more expensive) [Table 8] [DeepSeek-AI+ 2024]

每一个 token, 每一层, 要存一个 16384 维的 KV 向量 → KV cache 爆炸的根源。

RoPE (旋转位置编码) 是在 head 维度上做几何旋转 压缩到 latent 空间后, 位置信息不可逆 所以 DeepSeek 的做法是: 保留 64 维专门给 RoPE latent 部分 $512 + \text{RoPE } 64 = 576$ 这是一个工程妥协, 不是概念问题。

任务: BBH, MMLU, C-Eval, CMMLU 都是推理 / reasoning / hard benchmark。MHA 最好, GQA 次之, MQA 最差。而且是系统性差距, 不是噪声。在“需要复杂注意力结构”的任务上, KV 的表达能力是真的重要。

Benchmark (Metric)	# Shots	Dense 7B w/ MQA	Dense 7B w/ GQA (8 Groups)	Dense 7B w/ MHA
# Params	-	7.1B	6.9B	6.9B
BBH (EM)	3-shot	33.2	35.6	37.0
MMLU (Acc.)	5-shot	37.9	41.2	45.2
C-Eval (Acc.)	5-shot	30.0	37.7	42.9
CMMLU (Acc.)	5-shot	34.6	38.4	43.5

Table 8 | Comparison among 7B dense models with MHA, GQA, and MQA, respectively. MHA demonstrates significant advantages over GQA and MQA on hard benchmarks.

Second, MLA is a bit better than MHA (and much cheaper) [Table 9] [DeepSeek-AI+ 2024]

Benchmark (Metric)	# Shots	Small MoE w/ MHA	Small MoE w/ MLA	Large MoE w/ MHA	Large MoE w/ MLA
# Activated Params	-	2.5B	2.4B	25.0B	21.5B
# Total Params	-	15.8B	15.7B	250.8B	247.4B
KV Cache per Token (# Element)	-	110.6K	15.6K	860.2K	34.6K
BBH (EM)	3-shot	37.9	39.0	46.6	50.7
MMLU (Acc.)	5-shot	48.7	50.0	57.5	59.0
C-Eval (Acc.)	5-shot	51.6	50.9	57.9	59.2
CMMLU (Acc.)	5-shot	52.3	53.4	60.7	62.5

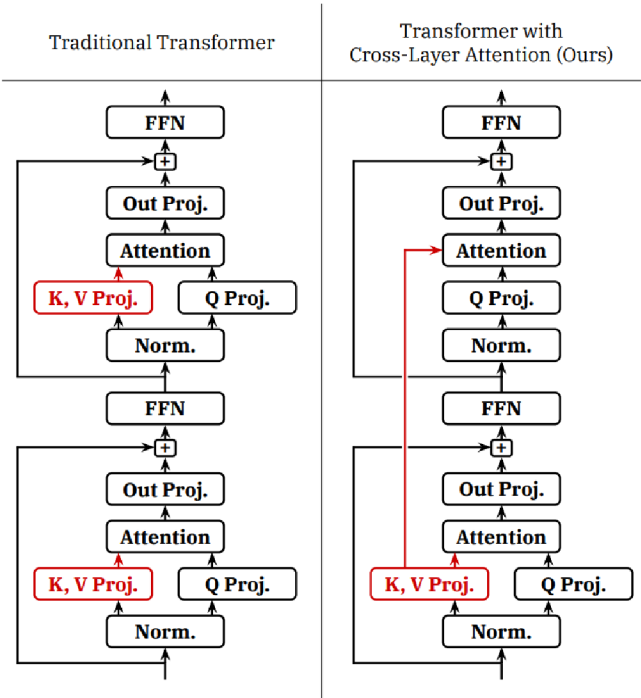
MLA的KV Cache少了很多 但是MLA preserve the accuracy of MHA 精度 $\text{MLA} \geq \text{MHA}$, 在多个 hard benchmark 上

344

Cross-layer attention (CLA)

[Brandon+ 2024]

345



Traditional Transformer Layer: $X \rightarrow \text{Norm} \rightarrow Q \text{ Proj} \rightarrow K \text{ Proj} \rightarrow V \text{ Proj} \rightarrow \text{Attention} \rightarrow \text{Out Proj} \rightarrow \text{FFN}$
每一层都有自己的一套 attention 每一层都会算出自己的一套 K 和 V KV cache 不是只存 token, 而是 "token $\times$ layer $\times$ (K,V)" 的一个三维结构. KV cache 大小 $\propto L \times S \times (KV \text{ 维度})$ Recall前面提到的: 在自回归生成 (generation) 时 每生成一个新 token attention 都要回头看之前所有 token 但你不能每一步都重新算一遍过去的 K/V (太慢) 所以把每一层算出来的 Key / Value 存起来 这块内存就叫 KV cache
KV cache = 对于每一层 $\ell = 1..L$ 对于每一个历史 token $t = 1..S$ 存 $K_{\ell}(t)$ 向量, $V_{\ell}(t)$ 向量 = [token position] $\times$ [layer] $\times$ {K, V}

Transformer with CLA: For CLA2 (sharing factor = 2) Layer 0: 计算 $K_0, V_0 \rightarrow$ cache Layer 1: $\times$ 不算 K/V , 用 Layer 0 的 K_0, V_0 Layer 2: 计算 $K_2, V_2 \rightarrow$ cache Layer 3: $\times$ 用 Layer 2 的 $K_2, V_2 \dots$ 不是每一层都做 K/V projection KV cache 大小 = 原来的 $1 / \text{sharing_factor}$ Traditional: 每一层都各自存一套 KV CLA2: 两层共用一套 KV CLA4: 四层共用一套 KV

GQA: 多个 query heads 共享同一组 K/V; CLA: 多个 Transformer layers 共享同一组 K/V

346

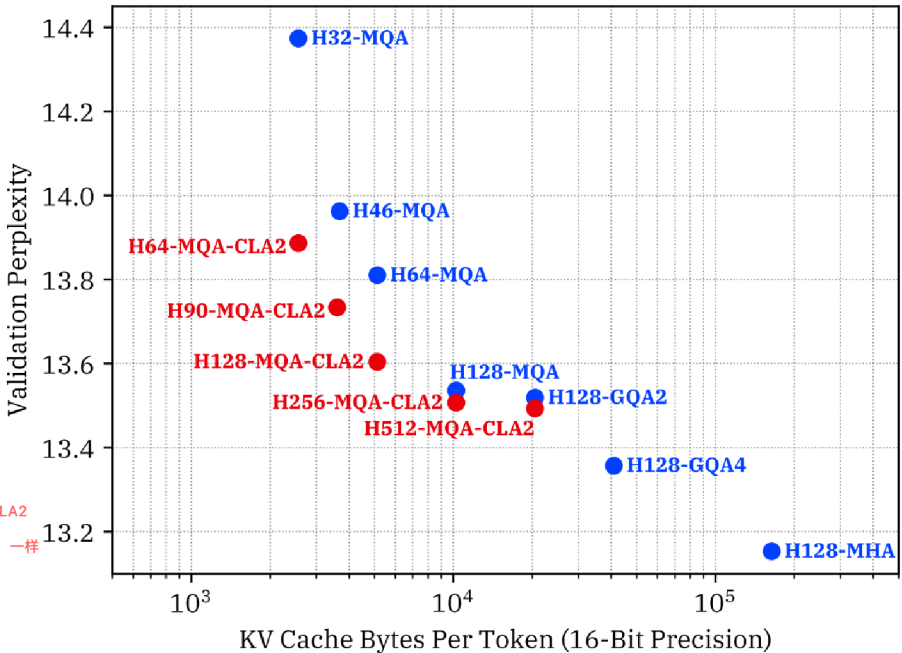
Idea: share KVs across layers (just as GQA shares KVs across heads)

347

Empirically improves the pareto frontier of accuracy and KV cache size (latency and throughput)

348

Pareto Frontier with and without CLA (1B Models)



recall defin of pareto frontier The pareto frontier represents the set of optimal trade-offs where improving one objective (eg. latency, quality) inevitably sacrifices another objective (eg. throughput, cost). the set of all Pareto optimal/efficient solutions. For Multi-Objective Optimization Problems with several conflicting goals (e.g., speed vs. power, cost vs. quality). feasible sets: the total range of possible choices Pareto Optimal/Efficient Solutions: Solutions on the frontier that are not dominated by any other solution (in the feasible set).

看图: H64-MQA和H128-MQA-CLA2 (head dim两倍, 但是cla sharing factor也是2) 的kv缓存(x轴横坐标) 一样

349

x axis: 每生成 1 个 token, KV cache 要占多少显存 (这是 inference 成本的核心瓶颈)

350

Local attention

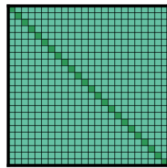
[Beltagy+ 2020][Child+ 2019][Jiang+ 2023]

为了让 Transformer 推理更快、更省显存 (KV cache 更小), 我们可以不让每个 token 看“所有历史”, 而是只看“附近的一小段上下文”

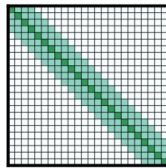
attention 本质是: Query 去“查阅”input sequence (Key / Value) input sequence = 已经存在、可被注意力读取的 token (K/V, 长度 S) query = 当前要计算的 token (Q, 长度 T) Q 是“我要问的问题” K/V 是“我可以参考的材料” input sequence就是K/V 那条序列(长度 S),也就是: - prompt 里的 token - 已经生成过的历史 token - 存在 KV cache 里的 token

大多数 token: 局部看 少数 token / 少数层: 全局看 L 有限 (不能无限堆) 梯度 / 信息会衰减 一些任务需要一步直达的全局依赖 所以 layers + 少量 Global layers

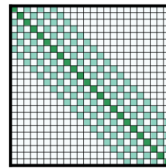
Full Attention: $kv_cache_size = S * (K * H) * L * 2 * 2$ 每个 token 都要存完整 K/V $O(S^2)$ s is length of input sequence



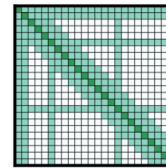
(a) Full n^2 attention



(b) Sliding window attention



(c) Dilated sliding window



(d) Global+sliding window

Idea: just look at the local context, which is most relevant for modeling

Effective context scales linearly with the number of layers

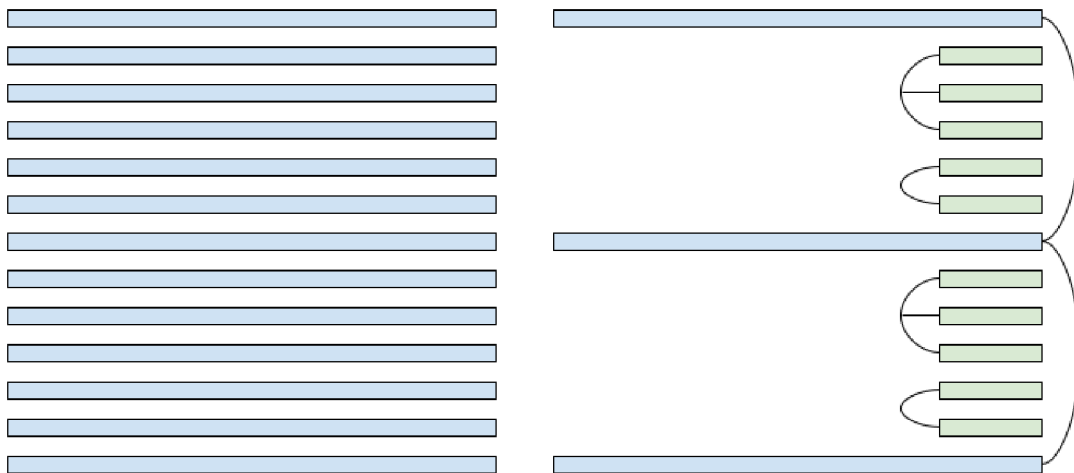
KV cache is independent of sequence length!

第 1 层: 每个 token 看 $\pm w$ 第 2 层: 它“间接”能拿到 $\pm 2w$ 第 L 层: $\pm(L \cdot w)$ 虽然单层是局部的, 但堆很多层后, 信息能慢慢“传很远” 每一层最多扩展 $\pm w$, L 层叠加, 信息“像波纹一样向外扩散” effective_context $\approx L \times w$

Problem: this can still hurt accuracy

Solution: interleave local attention with global attention (hybrid layers)

Example: character.ai uses 1 global layer every 6 layers (in addition to CLA) [article]



Summary:

- Goal: reduce the KV cache size (since inference is memory-limited) without hurting accuracy
- Lower-dimensional KV cache (GQA, MLA, shared KV cache)
- Local attention on some of the layers

def alternatives_to_the_transformer():

We have shown that tweaking the architecture of the Transformer, we can improve latency and throughput.

Attention + autoregression is fundamentally memory-limited (Transformers were not designed with inference in mind).

Can we substantially improve things if we go beyond the Transformer?

We will discuss two directions: state-space models and diffusion models.

State-space models

SSM (状态空间模型): 用 $O(1)$ 状态替代 $O(T)$ 缓存; 很多系统会和注意力混搭。

[presentation from CS229S]

- Idea: from signal processing to model long-context sequences in a sub-quadratic time
- S4: based on classic state space models, good at synthetic long-context tasks [Gu+ 2021]

State Space Models (SSM) 不是在“看所有历史 token”, 而是在“用一个状态向量, 持续记住过去的信息”。离散ssm:

$x_{t+1} = Ax_t + Bu$ (递归) 系统稳定 $\Leftrightarrow$ 谱半径 $\rho(A) < 1$, 当 $t \rightarrow \infty$, $x(t)$ 不会无限变大 连续ssm: $x'(t) = dx(t)/dt$

$= Ax(t)$ (常微分方程ode) 如果要系统稳定 $\Leftrightarrow$ 矩阵A 的所有特征值eigenvalue实部 < 0

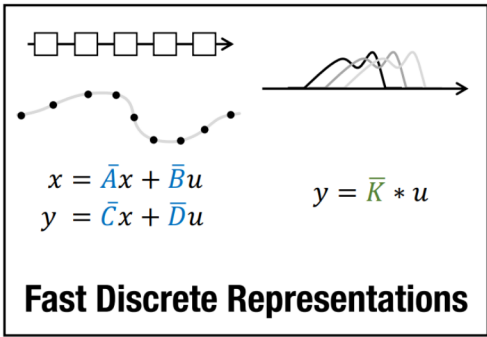
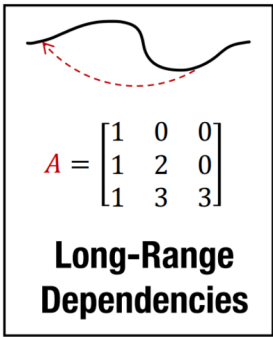
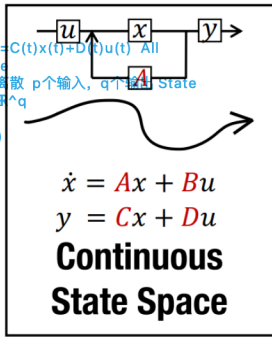
(ODE常微分方程/未知函数只含有一个自变量的微分方程, PDE用于多个自变量) ssm有markov property(x_{t+1} depends only on x_t and u_t), 但是ssm是deterministic而不是stochastic process, 没有随机性

S4 paper: Lemma - Conjugation is an equivalence relation on SSMs 如果两个 SSM 只是换了一个状态坐标/基底, 那么它们在行为上是完全一样的模型 (在同一个 state space 里, 维度不变, vector space不变) 对 state x 做一个可逆的线性变换: $x(t) = Vx(t)_{tilde}$, V 是可逆矩阵 代入ssm, 得出: $A \rightarrow V^{-1}AV, B \rightarrow V^{-1}B, C \rightarrow CV$. 新的ssm和原来的ssm是equivalent的. the naive application of diagonalization does not work due to numerical issues. we should only conjugate by well-conditioned matrices V. A 决定三件事: 记忆多久 (衰减decay快还是慢, 离散化比如Euler之后影响decay/retention系数), 稳定性 (会不会爆炸), 不同 state 维度如何相互混合. A 的eigenvalue决定了记忆多久 / 衰减多快: 特征值实部越负 $\rightarrow$ 记得越快; 越接近 0 $\rightarrow$ 记得越久 for S4, A: HiPPO / diagonal + low-rank form SSM $\rightarrow$ convolution $\rightarrow$ Cauchy kernel

The following model architectures struggle to scale to very long sequences of 10000 or more steps: CNN (路径长度 = 层数, 只能“一层一层”传信息”, 每一层只能扩大很小的上下文范围, 感受野Receptive Field增长慢), RNN (梯度消失 / 爆炸), Transformer (时间复杂度: $O(T^2)$ 二次增长, KV cache 大小: $O(T)$ 线性增长, 无法10k token) Alternatives: subquadratic-time architectures, ssm. 1. state space model (SSM) $x'(t) = Ax(t) + Bu(t)$, $y(t) = Cx(t) + Du(t)$ with appropriate choices of the state matrix A, SSM can handle long-range dependencies mathematically, but prohibitive computation and memory requirements 用 state 记住过去, 但太慢 / 太难算 (原始形式 2. Structured State Space sequence model (S4) conditioning A with a low-rank correction, allowing it to be diagonalized stably and reducing the SSM to the well-studied computation of a Cauchy kernel 60x faster comparing with Transformers on image and language modeling tasks. 让 SSM 可高效计算. s4里的矩阵A是固定的, 结构化的 (对角 + 低秩), 目的是稳定 (stability of the model); 把 SSM $\rightarrow$ 卷积; 用 FFT 高效算, 能跑 10k+ 序列. 长程很强, 但“不懂语义轻重”, 对语言这种“内容敏感”的任务不够灵活. 3. Mamba: Linear-Time Sequence Modeling with Selective State Spaces 让 SSM 对内容自适应. 参数矩阵A,B随输入变化 (input adaptive) $A_t = \text{diag}(\log A(u_t))$ 也就是 $x_{t+1} = A(u_t) \otimes x_t + (input \text{ term})$. 这里 $A(u_t)$ is decay, 是由当前 token 生成, 每个维度一个, 对角的 $\otimes$: 逐元素乘, letting the SSM parameters be functions of the input, allowing the model to selectively propagate or forget information along the sequence length dimension. Mamba out-performs prior state-of-the-art models such as S4, Hyena, and Transformers on modeling audio waveforms and DNA sequences, both in pretraining quality and downstream metrics its performance improves with longer context up to million-length sequences 4. Mamba [混合计算/存储] Built at large scale, Mamba provides high throughput and small memory footprint compared to vanilla Transformers, and at the same time state-of-the-art performance on standard language model benchmarks and long-context evaluations. Transformer: 用在少数层, 负责衔接 token 交互. Mamba: 用在大多数层, 负责长程记忆 + 高效推理. KV cache 大幅减少, inference 快, accuracy 不掉 (甚至更好)

377

General form of SSM: $x'(t)=A(t)x(t)+B(t)u(t)$ $y(t)=C(t)x(t)+D(t)u(t)$ All matrices are time variant or time invariant 形式: 是否时变, 连续或离散 p 个输入, $q+1$ 个 State $x \in \mathbb{R}^n$ Input $u \in \mathbb{R}^p$ output $y \in \mathbb{R}^q$ $x'(t)=dx(t)/dt$ $A \in \mathbb{R}^{(n \times n)}$ $B \in \mathbb{R}^{(n \times p)}$ $C \in \mathbb{R}^{(q \times n)}$ $D \in \mathbb{R}^{(q \times p)}$



Left: State Space Models (SSM) parameterized by matrices A, B, C, D map an input signal $u(t)$ to output $y(t)$ through a latent state $x(t)$. Center: Recent theory on continuous-time memorization derives special 'A' matrices that allow SSMs to capture LRDs mathematically and empirically. Right: SSMs can be computed either as a recurrence (left) or convolution (right). However, materializing these conceptual views requires utilizing different representations of its parameters (red, blue, green) which are very expensive to compute. S4 introduces a novel parameterization that efficiently swaps between these representations, allowing it to handle a wide range of tasks, be efficient at both training and inference, and excel at long sequences.

Kalman Filter

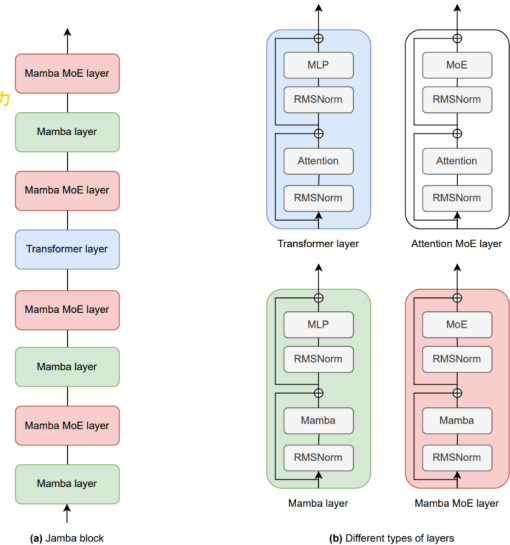
- Weaknesses: bad at solving associative recall tasks important for language (where Transformers do well)

A 4 B 3 C 6 F 1 E 2 $\rightarrow$ A ? C ? F ? E ? B ?
Key-Value Query

5x higher throughput than Transformers

- Mamba: allow SSM parameters to be input-dependent, match Transformers at 1B scale [Gu+ 2023]
- Jamba: interleave Transformer-Mamba layers (1:7 ratio) with a 52B MoE [Lieber+ 2024]

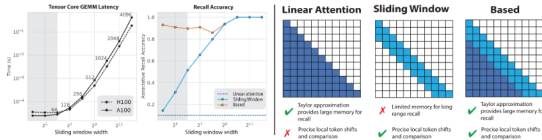
ratio 1:7 不是每一层都需要全局注意力



ssm+transformer Transformer 的强项 精确的 token-to-token 关系 全局依赖 (尤其是短到中等长度) 表达能力强 Transformer 的弱点 KV cache 很大 推理阶段 memory-limited 超长序列 (10k+) 代价高 SSM / Mamba 的强项 用 state 记忆历史 O(1) memory 长序列稳定 inference 非常快 SSM / Mamba 的弱点 精细 token 交互能力不如 attention 局部对齐、精确拷贝能力稍弱

SSM 的 A, B, C, D 都是通过反向传播学习的; 如果像普通神经网络一样, 用 loss + backprop, 会出问题: 1) 不稳定, A 的特征值可能 > 0 或 |λ| > 1, 状态爆炸 / 消失 2) 计算慢 递推是 O(T*N^2) 3) 长序列梯度爆炸 / 消失 所以 S4 / Mamba 的全部创新, 几乎都在“怎么学 A, B, C, D” 真正的挑战不是“能不能学”, 而是如何参数化 A 才能: 1. 稳定 (eigenvalue |λ| < 1) 2. 可计算 (A 必须能高效算 A^n) 3. 能建模长依赖 (记忆让 |λ|=1 接近于 1)。 S4 用结构化线性代数解决, A=A-PQ^T A: 对角矩阵 (主动力学) P, Q: low rank 可训练参数是: A, P, Q, B, C, D FFT (Fast Fourier Transform, 快速傅里叶变换) 是一种把“时域里的序列运算 (尤其是 O(T^2) 到 O(T log T) 的算法。 example: y_t 来自当前以及前两步输入, 那么 y_t=h_0u_t+h_1u_{t-1}+h_2u_{t-2} h 卷积核 kernel, y 输出序列 卷积 y=h*u $\Leftrightarrow$ 傅立叶变换 F(y)=F(h)*F(u) 为什么 S4 要花那么大力气设计 A 因为卷积核 kernel 变(A^n)影响: h=h_n=C(A^n) B (对离散的ssm模型推导消去, derive y和u的关系: y_t=sum(h_n*u_n), 得出h_n=C(A^n) B) 卷积核 h_n=“输入在 n 步之前, 对当前输出的影响” A 的特征值=h 的衰减速度 如果 A 的特征值: 接近 1 $\rightarrow$ 核很长 $\rightarrow$ 长程记忆 很小 $\rightarrow$ 核很短 $\rightarrow$ 快速遗忘 复数 $\rightarrow$ 核振荡 $\rightarrow$ 结构性模式 设计 A = 设计卷积核形状 S4: 超长(长程记忆)、结构性核 (FFT 算), 但 S4 的矩阵是 1xk 的, 无论输入是什么, 模型“如何记忆过去”的 OK 的 (信号、时间序列、合成任务), 但在 language 模型里是硬伤。语言: 有些 token (eg. not, if) 值得记住 1000 步, 有些 token (eg. the) 看一眼就忘。 FFT需要整段序列, 不适合 generation, O(T log T)。而 Mamba input dependent ssm, 用 input-adaptive recurrence 解决。位置相关、动态核, 不需要 FFT, 不需要缓存 KV, O(1) per token。

- BASED: use linear attention + local attention [Arora+ 2024]



- MiniMax-01: use linear attention + full attention (456B parameter MoE) [MiniMax+ 2025]

Takeaways:

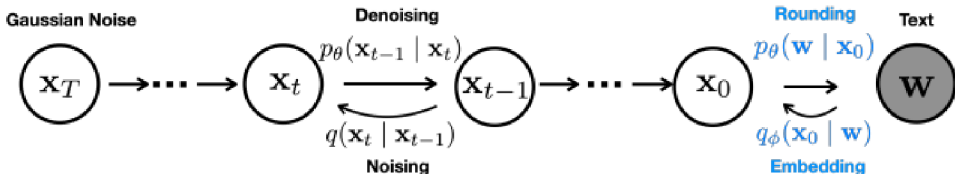
- Linear + local attention (still need some full attention) yield serious SOTA models
- Replace O(T) KV cache with O(1) state => much more efficient for inference

Li (2022): Diffusion-LM iteratively denoises a sequence of Gaussian vectors into word vectors, yielding a sequence of intermediate latent variables. Motivation: Controlling attributes of the generated language (e.g. switching topic or sentiment) without re-training, updating the LM parameters for each control task can be expensive and does not allow for compositions of multiple controls.

Diffusion models

Stochastic Differential Equations (SDEs)

- Popular for image generation, but harder to get working for text generation [Li+ 2022]



Diffusion 的时间步 t 不是 token 位置, 而是噪声级别/去噪迭代步数。

- Idea: generate each token in parallel (not autoregressively), refine multiple time steps
- Start with random noise (over entire sequence), iteratively refine it

Note: diffusion 是 stochastic 学习一个“如何从 noisy 样本回到更干净样本”的随机逆过程 非自回归, 整个序列同时生成, 多次采样 $\rightarrow$ 多样解 适合图像, 音频, 并行生成 语言是离散 + 强因果顺序, 而 diffusion 擅长连续、全局并行 也有研究关于 text diffusion models on discrete state spaces 这篇 paper focus on continuous diffusion models for text text 是离散的, 但 diffusion 在 $\mathbb{R}^d$ 里工作 token $\rightarrow$ embedding (连续) diffusion 在 embedding space 做最后: rounding / decoding 回 token

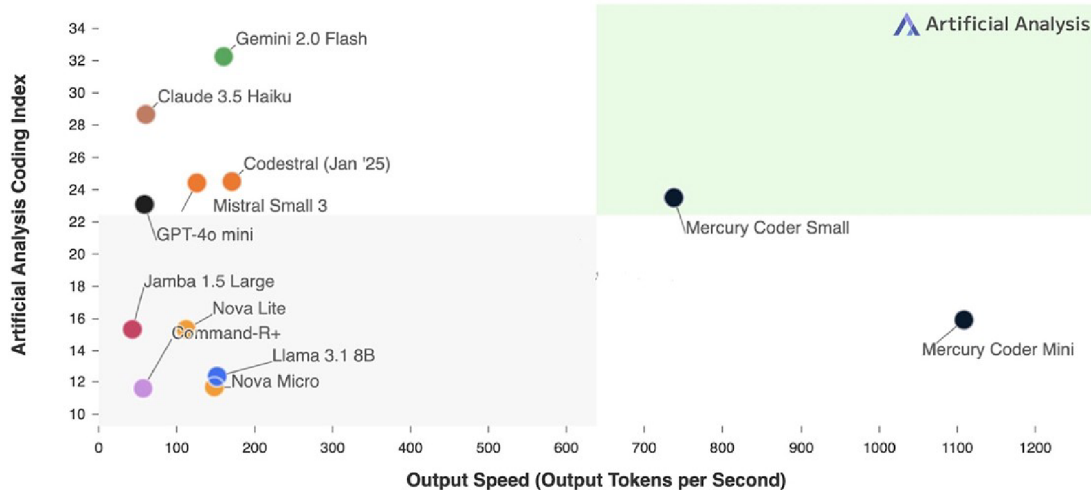
Figure: A graphical model representing the forward and reverse diffusion processes. In addition to the original diffusion models, we add a Markov transition between x_0 and w , and propose the embedding §4.1 and rounding §4.2 techniques in the paper.

Autoregressive models: error propagation, indirect control, token by token generation Diffusion models: iterative refinement, explicit control, faster sampling refinement 所有 token 都可以反复被修改, 错误传播可被修正, token 全并行。

- Results from Inception Labs [article]
[demo video]

Much faster on coding benchmarks:

越高越好



Overall, significant gains in inference to be made with more radical architecture changes!

量化 = 用更少的比特 (更低精度) 来存数字, →
更少内存读写 → 推理更快 (因为 LLM 推理是
memory-bound) → 但要小心精度损失

def quantization():

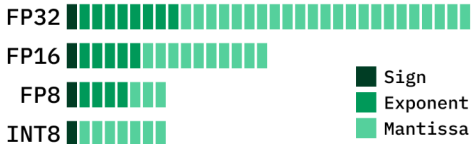
Key idea: reduce the precision of numbers

Less memory means higher latency/throughput (since inference is memory-limited).

Of course we have to worry about accuracy...

Comparing number formats

[article]



FP8: 浮点数 Floating Point, 8 bits, 最大可用数值范围 [-240, 240], 例如 H100 常用的 FP8
格式 E4M3: sign1 | exponent4 | mantissa3, 用 exponent
换来了更大的动态范围, 但不精确 INT8: 有符号整数, 范围 = $[-2^7, 2^7 - 1] = [-128, 127]$
只能表示 256 个离散值, 配合 scale INT4: 范围 = $[-2^3, 2^3 - 1]$, 只有 16
个数 [-8, 7], 精度非常粗, 因为区间太小。配合 scale 缩放才能用, scale
把真实数“映射”到可表示区间。位数越少 → 精度越差, 但内存占用越小

每一个数存三样: sign | exponent | mantissa
Sign (符号位): 正 / 负
Exponent (指数): 范围有多大
Mantissa (尾数): 精度有多细

- fp32 (4 bytes): needed for parameters and optimizer states during training 精度最高, 训练用
- bf16 (2 bytes): default for inference bf16 / fp16: 推理主流
- fp8 (1 byte) [-240, 240] for e4m3 on H100s: can train if you dare [Peng+ 2023]
- int8 (1 byte) [-128, 127]: less accurate but cheaper than fp8, but for inference only [Baaen+ 2023]
- int4 (0.5 bytes) [-8, 7]: cheaper, even less accurate [Baaen+ 2023]

Quantization-aware training (QAT): train with quantization, but doesn't scale up 需要重新训练, 算力成本高

Post-training quantization (PTQ): run on sample data to determine scale and zero point for each layer or tensor

[Overview of approaches]

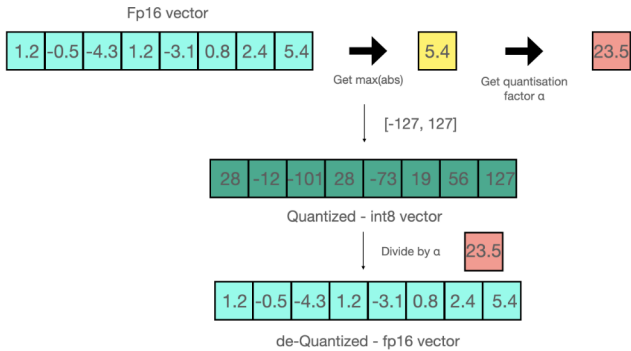
PTQ: 模型已经训练完了, 再把 FP16 / FP32 的权重和激活压缩成 INT8 / INT4 / FP8, 用于推理 The fundamental challenge lies in performing this mapping with minimal loss of information. (prevent overflow underflow)

LLM.int8()

[Dettmers+ 2022][article]

Standard quantization (scale by max of absolute values):

423



424
425
426

Problem: outliers (which appear in larger networks) screw everything up
Solution: extract outliers and process them in fp16



427
428

It works well (but is 15-23% slower than fp16):

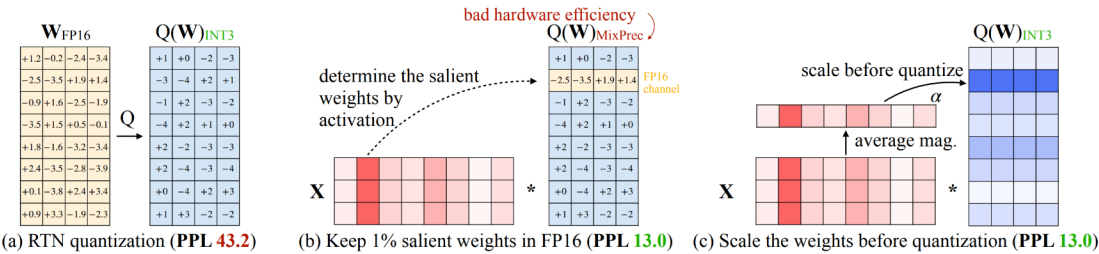
benchmarks	-	-	-	-	difference - value
name	metric	value - int8	value - bf16	std err - bf16	-
hellaswag	acc_norm	0.7274	0.7303	0.0044	0.0029
hellaswag	acc	0.5563	0.5584	0.005	0.0021
piqa	acc	0.7835	0.7884	0.0095	0.0049
piqa	acc_norm	0.7922	0.7911	0.0095	0.0011
lambada	ppl	3.9191	3.931	0.0846	0.0119
lambada	acc	0.6808	0.6718	0.0065	0.009
winogrande	acc	0.7048	0.7048	0.0128	0

429
430

Activation-aware quantization

431
432
433
434

[Lin+ 2023]
Idea: select which weights (0.1-1%) to keep in high precision based on activations
fp16 -> int3 produces 4x lower memory, 3.2x speedup



435
436

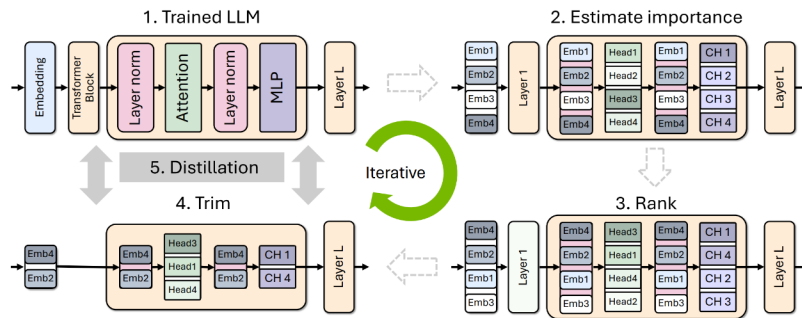
```
437 def model_pruning():
```

```
438     Key idea: just rip out parts of an expensive model to make it cheaper
439     ...and then fix it up.
```

```
440
```

```
441     Paper from NVIDIA [Muralidharan+ 2024]
```

```
442
```



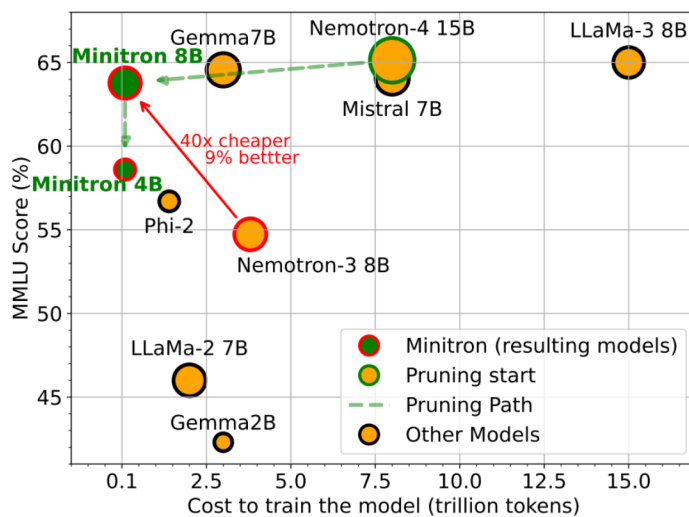
```
443     Algorithm:
```

- ```
444 1. Identify important {layer, head, hidden dimension} on a small calibration dataset (1024 samples)
445 2. Remove unimportant layers to get a smaller model
446 3. Distill the original model into pruned model
```

```
447
```

```
448 Results:
```

```
449
```



```
450
```

```
451
```

```
452 def speculative_sampling():
```

```
453 Recall the two stages of inference:
```

- ```
454     • Prefill: given a sequence, encode tokens in parallel (compute-limited) [note: also gives you probabilities]
455     • Generation: generate one token at a time (memory-limited)
```

```
456     In other words, checking is faster than generation.
```

“验证比生成快”：小草稿模型先猜几个 token，大模型并行检查并决定接受/修正；理论上能保证最终分布等于大模型

```
457
```

```
458     Speculative sampling [Leviathan+ 2022][Chen+ 2023]
```

- ```
459 • Use a cheaper draft model p to guess a few tokens (e.g., 4)
460 • Evaluate with target model q (process tokens in parallel), and accept if it looks good
```

```
461 [Speculative sampling video]
```

```
462 [article]
```

```
463
```

464

**Algorithm 2** Speculative Sampling (SpS) with Auto-Regressive Target and Draft Models

Given lookahead  $K$  and minimum target sequence length  $T$ .

Given auto-regressive target model  $q(\cdot|\cdot)$ , and auto-regressive draft model  $p(\cdot|\cdot)$ , initial prompt sequence  $x_0, \dots, x_t$ .

Initialise  $n \leftarrow t$ .

**while**  $n < T$  **do**

**for**  $t = 1 : K$  **do**

    Sample draft auto-regressively  $\tilde{x}_t \sim p(x|x_1, \dots, x_n, \tilde{x}_1, \dots, \tilde{x}_{t-1})$

**end for**

  In parallel, compute  $K + 1$  sets of logits from drafts  $\tilde{x}_1, \dots, \tilde{x}_K$ :

$$q(x|x_1, \dots, x_n), q(x|x_1, \dots, x_n, \tilde{x}_1), \dots, q(x|x_1, \dots, x_n, \tilde{x}_1, \dots, \tilde{x}_K)$$

**for**  $t = 1 : K$  **do**

    Sample  $r \sim U[0, 1]$  from a uniform distribution.

**if**  $r < \min\left(1, \frac{q(x|x_1, \dots, x_{n+t-1})}{p(x|x_1, \dots, x_{n+t-1})}\right)$ , **then**

      Set  $x_{n+t} \leftarrow \tilde{x}_t$  and  $n \leftarrow n + 1$ .

**else**

      sample  $x_{n+t} \sim (q(x|x_1, \dots, x_{n+t-1}) - p(x|x_1, \dots, x_{n+t-1}))_+$  and exit for loop.

**end if**

**end for**

  If all tokens  $x_{n+1}, \dots, x_{n+K}$  are accepted, sample extra token  $x_{n+K+1} \sim q(x|x_1, \dots, x_n, x_{n+K})$  and set  $n \leftarrow n + 1$ .

**end while**

465

This is modified rejection sampling with proposal  $p$  and target  $q$

466

Modification: always generate at least one candidate (rejection sampling will keep looping)

467

Key property: guaranteed to be an **exact sample** from the target model!

468

469

Proof by example: assume two vocabulary elements  $\{A, B\}$

470

- Target model probabilities:  $[q(A), q(B)]$

471

- Draft model probabilities:  $[p(A), p(B)]$

472

- Assume  $p(A) > q(A)$  [draft model oversamples A].

473

- Therefore  $p(B) < q(B)$  [draft model undersamples B].

474

- Residual probabilities  $\max(q-p, 0)$ :  $[0, 1]$

475

Compute the probabilities of speculatively sampling a token:

476

- $P[\text{sampling } A] = p(A) * (q(A) / p(A)) + p(B) * 1 * 0 = q(A)$

477

- $P[\text{sampling } B] = p(B) * 1 + p(A) * (1 - q(A) / p(A)) * 1 = q(B)$

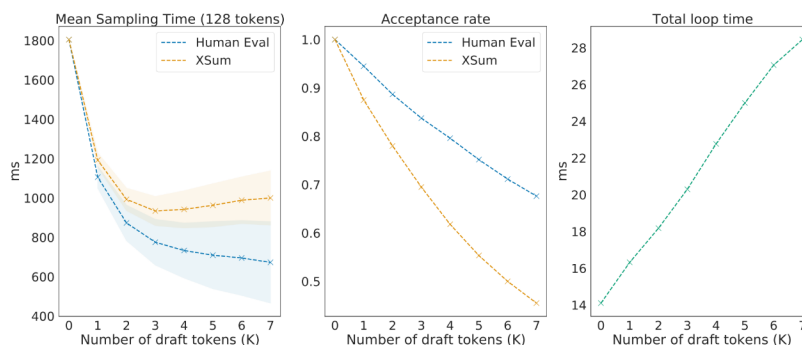
478

479

Table 1 | **Chinchilla performance and speed on XSum and HumanEval with naive and speculative sampling at batch size 1 and  $K = 4$ .** XSum was executed with nucleus parameter  $p = 0.8$ , and HumanEval with  $p = 0.95$  and temperature 0.8.

| Sampling Method | Benchmark            | Result | Mean Token Time | Speed Up |
|-----------------|----------------------|--------|-----------------|----------|
| ArS (Nucleus)   | XSum (ROUGE-2)       | 0.112  | 14.1ms/Token    | 1×       |
| SpS (Nucleus)   |                      | 0.114  | 7.52ms/Token    | 1.92×    |
| ArS (Greedy)    | XSum (ROUGE-2)       | 0.157  | 14.1ms/Token    | 1×       |
| SpS (Greedy)    |                      | 0.156  | 7.00ms/Token    | 2.01×    |
| ArS (Nucleus)   | HumanEval (100 Shot) | 45.1%  | 14.1ms/Token    | 1×       |
| SpS (Nucleus)   |                      | 47.0%  | 5.73ms/Token    | 2.46×    |

480



481

482

In practice:

483

- Target model has 70B parameters, draft model has 8B parameters

484

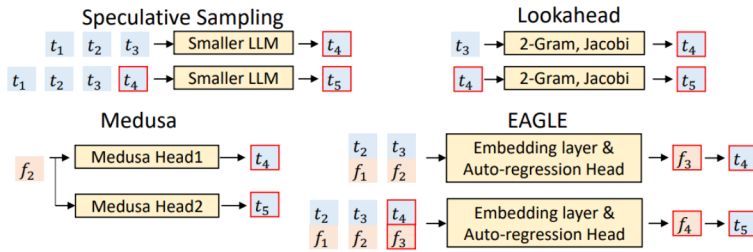
- Target model has 8B parameters, draft model has 1B parameters

485

- Try to make draft model as close to target (distillation)

Extensions to improve the draft model:

- Medusa: draft model generates multiple tokens in parallel [Cai+ 2024]
- EAGLE: draft model takes high-level features from target model [Li+ 2024]



Summary:

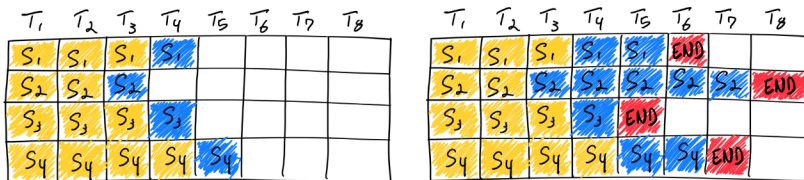
- Exact sampling from target model (thanks to math)!
- Exploits asymmetry between checking and generation
- Lots of room for innovation on the draft model (involves training)

```
def continuous_batching():
```

Orca: A Distributed Serving System for Transformer-Based Generative Models[talk]

Problem:

- Training: get a dense block of tokens (batch size x sequence length)
- Inference: requests arrive and finish at different times, so you have a ragged array



**Solution: iteration-level scheduling**

- Decode step by step
- Add new requests to the batch as they arrive (so don't have to wait until generation completes)

Problem:

- Batching only works when all sequences have the same dimensionality (right?)
- But each request might have a different length

**Solution: selective batching**

- Training: when all sequences of the same length, operate on a  $B \times S \times H$  tensor
- But we might have different lengths:  $[3, H]$ ,  $[9, H]$ ,  $[5, H]$ , etc.
- Attention computation: process each sequence separately
- Non-attention computation: concatenate all the sequences together to  $[3 + 9 + 5, H]$

```
def paged_attention():
```

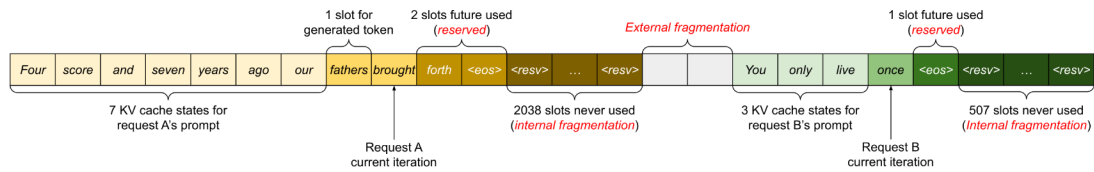
Paper that introduced vLLM in addition to PagedAttention [Kwon+ 2023]

Previous status quo:

- Request comes in
- Allocate section of KV cache for prompt and response (up to a max length)



527



528

**Problem: fragmentation (what happens to your hard drive)**

529

- But this is wasteful since we might generate much fewer tokens (internal fragmentation)!

530

- Might be extra unused space between sections (external fragmentation)!

531

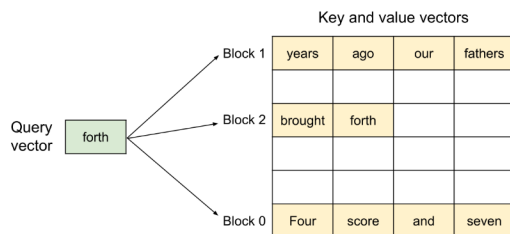
532

**Solution: PagedAttention (remember operating systems)**

533

- Divide the KV cache of a sequence into non-contiguous **blocks**

534

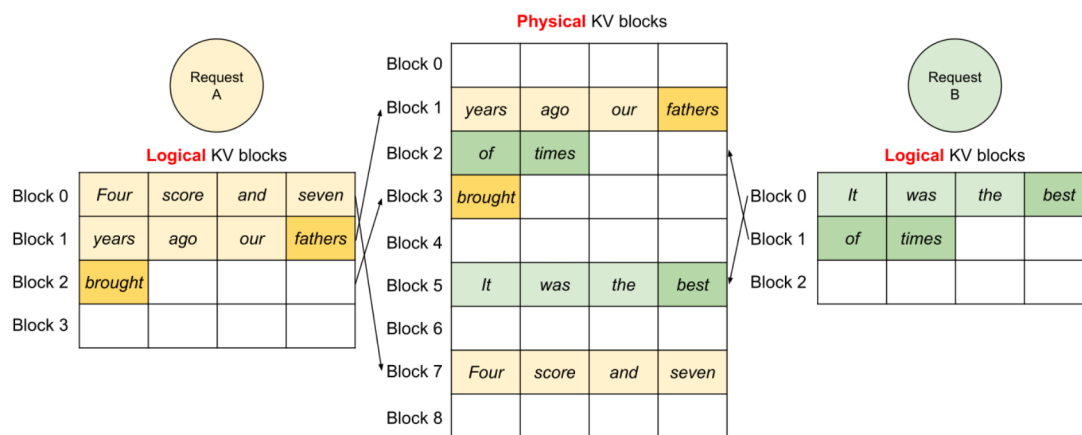


535

536

Two requests share the KV caches:

537

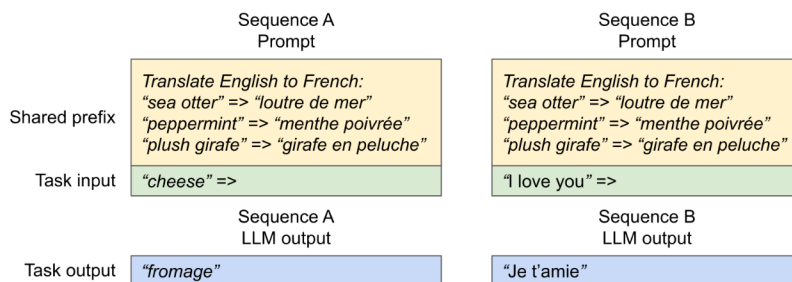


538

539

In general, multiples types of sharing KV caches across sequences:

540



541

- Sharing the system prompt

542

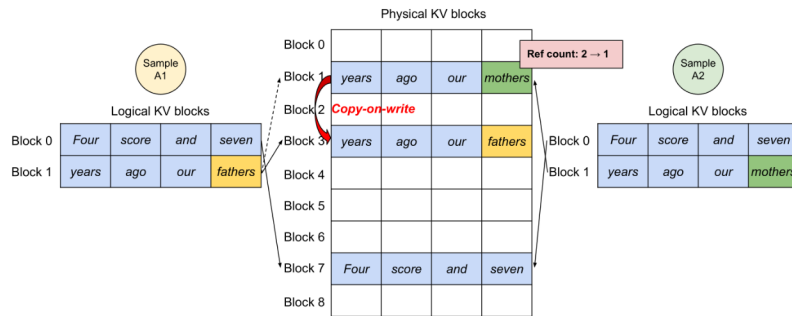
- Sampling multiple responses per prompt (e.g., for program synthesis)

543

544

Solution: share prefixes, copy-on-write at the block level

545



546

547

Other vLLM optimizations:

548

- Kernel to fuse block read and attention (reduce kernel launch overhead)

549

- Use latest kernels (FlashAttention, FlashDecoding)

550

- Use CUDA graphs to avoid kernel launch overhead

551

552

Summary: use ideas from operating systems (paging) to make use of memory for dynamic workloads

553

554

555 `if __name__ == "__main__":`556 `main()`